

Hausaufgabe 4: Aufgabe 3 - Interrupts

3. Interrupts (Basierend auf Vorlesung 11)

1. Welche Arten von Interrupts gibt es? Wodurch werden diese ausgelöst?

- Tastatur, Maus;
- Timer.

2. Wie bzw. wann wird auf das Eintreten eines Interrupts reagiert? Wie unterscheiden sich die Arten hier?

Wann: Control detektiert die Ausnahme **nach einer Instruktion (bzw. parallel dazu, vgl. Idee 11.2, Folie 16)**. Die Reaktion muss die **Transparenz-Forderung (Folie 23)** erfüllen: Das laufende Programm darf nichts bemerken.

- PC zeigt auf aktuelle Instruktion.
- Register und Stapel (Stack) sind unverändert.
- Es vergeht lediglich Zeit.

Unterschiede: Die ISR (Interrupt Service Routine) ist im Wesentlichen ein Unterprogramm, das durch Control mittels Mikroinstruktionen angestoßen wird (Folie 24). Die Arten (z.B. Timer vs. I/O) unterscheiden sich hauptsächlich durch ihren Eintrag in der **Interrupt Table** (Folie 25), wodurch die passende ISR angesprungen wird.

3. Verantwortlichkeiten & Rückgabewerte

Verantwortlichkeiten (vgl. Folie 27 & 30): Da der Aufruf durch die Hardware erfolgt, gibt es keinen klassischen „Aufrufer“, der Register sichert.

1. **Hardware (Control):** Übernimmt teilweise Aufgaben, insbesondere **das Retten des PC (Program Counter)**.
2. **Software (ISR):** Übernimmt den Rest, insbesondere das Sichern aller verwendeten Register und das Aufräumen des Stacks.

Rückgabewerte (vgl. Folie 29): ISRs haben **nie** einen Rückgabewert. Grund: Das unterbrochene Programm weiß nichts von der Unterbrechung. Ein Rückgabewert würde den Programmablauf nur verwirren („Wohin auch damit?“).

4. Nutzen von Sperrung und Priorisierung

(Die Vorlesung erwähnt explizit, dass Interrupts „abgeschaltet“ werden können (Folie 17) und dass Timer-Interrupts für Time-Sharing genutzt werden (Folie 34)).

Der Nutzen ist vielfältig:

- **Time-Sharing:** Ermöglicht Multitasking, indem Timer-Interrupts periodisch den Prozess wechseln (Folie 33-34).
- **Konsistenz:** Verhindert, dass eine ISR eine andere unterbricht, wenn dies zu Problemen führen würde.
- **Schutz:** Kritische Programmabschnitte können „atomar“ ausgeführt werden.

5. Interruptbehandlung via Sprungtabelle

a) Sicherheitsrisiko (Ungeschützte Tabelle)

Risiko: Ein Angreifer könnte die Sprungtabelle manipulieren, sodass normaler Code die Kontrolle übernimmt, aber mit erhöhten Rechten läuft (Folie 36).

Differenzierung (Folie 37): Es ist eine „Zwei-Klassen-Gesellschaft“ im Prozessor nötig:

1. **Normaler Modus (User Mode):** Eingeschränkte Rechte, kein Schreibzugriff auf Interrupt-Tabelle.

2. **Privilegierter Modus (Supervisor/Kernel Mode):** Darf alles.

Der Wechsel erfolgt automatisch bei einer Ausnahme (Folie 39).

b) **Prozess der Ausnahmebehandlung (Schematisch)**

Der Ablauf entspricht exakt **Abbildung 11.5 (Folie 28)**:

1. **Ausnahme passiert** (Trigger).
2. **Control (Hardware):**
 - Detektiert Ausnahme via Mikroinstruktion.
 - Sichert PC (+4).
 - Sucht passende ISR in der Tabelle.
 - Springt zur ISR.
3. **ISR (Software):**
 - `create_frame()`: Register sichern, Stackplatz schaffen.
 - Führt eigentliche ISR-Logik aus (`isr1`, `isr2...`).
 - `destroy_frame()`: Register wiederherstellen.
4. **Rücksprung:** ISR endet (ohne Rückgabewert).

c) **Wie endet die Behandlung? Was muss der Prozessor hier leisten?**

Die Behandlung endet mit einer speziellen Instruktion, um Sonderrechte aufzugeben (Folie 39). Bei RISC-V ist dies der Befehl **sret (Supervisor Exception Return)** (vgl. Folie 40).

Aufgabenteilung (Wichtig!):

- **Die ISR (Software)** stellt die Register und den Stack wieder her (vgl. `destroy_frame` auf Folie 28).
- **Der Prozessor (Hardware)** leistet bei `sret`:
 1. Wechselt vom privilegierten Modus zurück in den normalen Modus.
 2. Lädt den gesicherten PC zurück, um das Programm fortzusetzen.